UCS420: Cognitive Computing

Assignment 2 — Python Data Structures
(Lists, Tuples, Sets, Dictionaries)

Important: The assignment requires your own roll number. The solution below is written so you only need to replace the value of `ROLL_NO` and your personal dictionary details. No hard-coded roll-number output is used.

Question 1. List operations

```
# Enter your own 10-digit roll number here
ROLL_NO = "YOUR_ROLL_NUMBER"

digits = [int(d) for d in ROLL_NO]
L = [d * 10 for d in digits]

# i. Print L
print("Initial L:", L)

# ii. append() adds an element at the end.
L.append(55)
print("After append(55):", L)

# insert(position, value) puts an element at the chosen index.
L.insert(2, 25)
print("After insert(2, 25):", L)

# iii. remove(value) removes the first matching value.
L.remove(55)
print("After remove(55):", L)

# pop() removes and returns an element; here we remove the last one.
removed = L.pop()
print("Popped element:", removed)
print("After pop():", L)

# iv. Sort ascending, then descending.
L.sort()
print("Ascending:", L)

L.sort(reverse=True)
print("Descending:", L)

# v. Slicing
print("First three:", L[:3])
print("Last three:", L[-3:])

# vi. List comprehension using the average.
average = sum(L) / len(L)
above_average = [x for x in L if x > average]
print("Average:", average)
print("Above average:", above_average)
```

• append() changes the list by adding the value at the end, while insert() adds it at a specified index.

• remove() deletes by value; pop() deletes by index (the default is the last element) and returns the removed element.

• sort() changes the list in place. sort(reverse=True) gives descending order.

• Slicing L[:3] gives the first three elements and L[-3:] gives the last three.

• The comprehension keeps only values strictly greater than the computed average.

Question 2. Tuple operations

```
# Use the first 8 values from L from Question 1.
# This line assumes L has already been created.
scores = tuple(L[:8])

# i. Highest score + index; lowest score + frequency
highest = max(scores)
```

```
highest_index = scores.index(highest)
lowest = min(scores)
lowest_count = scores.count(lowest)

print("Scores:", scores)
print("Highest:", highest, "Index:", highest_index)
print("Lowest:", lowest, "Count:", lowest_count)

# ii. Reverse tuple and return the result as a list
reversed_scores = list(reversed(scores))
print("Reversed as list:", reversed_scores)
# Tuple is immutable, so it cannot be reversed in place.

# iii. Search for a user-entered score
user_score = int(input("Enter a score: "))
if user_score in scores:
    print("First occurrence index:", scores.index(user_score))
else:
    print("Score is not present in the tuple.")

# iv. Attempt direct modification
try:
    scores[0] = 100
except TypeError as e:
    print("Error:", e)
    # Tuples are immutable; lists allow item assignment.

# v. Unpacking using *
first, second, *remaining = scores
print("First:", first)
print("Second:", second)
print("Remaining:", remaining)
```

- Tuples are immutable: their existing elements cannot be reassigned, unlike list elements.
- reversed(scores) returns an iterator, so list(reversed(scores)) creates a reversed list.
- index() returns the first occurrence, while count() returns the number of occurrences.
- The statement first, second, *remaining = scores performs extended unpacking.

Question 3. Random numbers, odd/even/prime and frequency

```
import random
from collections import Counter

# Use your own roll number as the random seed.
ROLL_NO = "YOUR_ROLL_NUMBER"
random.seed(int(ROLL_NO))

# i. 100 random integers from 100 to 900 inclusive
numbers = [random.randint(100, 900) for _ in range(100)]
print("100 random numbers:", numbers)

# Prime-checking function
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

# ii. Odd numbers
odd_numbers = [n for n in numbers if n % 2 != 0]
print("Odd count:", len(odd_numbers))
print("Odd numbers:", odd_numbers)

# iii. Even numbers
even_numbers = [n for n in numbers if n % 2 == 0]
print("Even count:", len(even_numbers))
print("Even numbers:", even_numbers)

# iv. Prime numbers + actual prime list using list comprehension
prime_numbers = [n for n in numbers if is_prime(n)]
print("Prime count:", len(prime_numbers))
print("Prime numbers:", prime_numbers)

# v. Most frequent number and its frequency
counts = Counter(numbers)
```

```
most_frequent, frequency = counts.most_common(1)[0]
print("Most frequent number:", most_frequent)
print("Frequency:", frequency)
```

• random.seed(int(ROLL_NO)) makes the random sequence reproducible for the same roll number; a different seed gives a different sequence.

• randint(100, 900) includes both endpoints.

• Every integer is classified as odd or even, so odd count + even count = 100.

• For primality, testing divisors only up to sqrt(n) is sufficient.

Question 4. Set operations

```
# Use the same 8 digits of the roll number BEFORE multiplying by 10.
ROLL_NO = "YOUR_ROLL_NUMBER"
digits = [int(d) for d in ROLL_NO[:8]]

A = {digit * 7 for digit in digits}
B = {digit * 9 for digit in digits}

print("A =", A)
print("B =", B)

# vi. Union: all unique values in A or B
print("A union B =", A.union(B))

# vii. Intersection: values common to both
print("A intersection B =", A.intersection(B))

# viii. Separate differences
print("A - B =", A.difference(B))
print("B - A =", B.difference(A))
# difference() keeps values only in the first set; symmetric_difference()
# keeps values that are in either set but not in both.

# ix. Symmetric difference
print("Symmetric difference =", A.symmetric_difference(B))

# x. Subset and superset checks
print("A subset of B:", A.issubset(B))
print("B superset of A:", B.issuperset(A))

# xi. discard() safely removes X if it exists
X = int(input("Enter X to remove from A: "))
A.discard(X)
print("A after discard:", A)
# discard() does nothing if X is absent; remove() would raise KeyError.
```

• Sets store unique elements, so repeated digits can produce fewer set elements than the number of digits.

• difference() is directional: A−B is not necessarily the same as B−A.

• symmetric_difference() combines both one-sided differences and excludes common values.

• issubset() checks whether every element of A is in B; issuperset() checks the reverse relationship.

• discard() is safer when you are unsure whether the value exists because it does not raise an error for a missing element.

Question 5. Dictionary operations

```
# Replace the placeholders with your own details.
my_dict = {
    "name": "<your name>",
    "roll_no": "<your roll number>",
    "branch": "<your branch>",
    "age": <your age>,
    "city": "<your home city>"
}

# i. Rename city -> location without rebuilding the whole dictionary
my_dict["location"] = my_dict.pop("city")
print("After renaming city:", my_dict)

# ii. Add CGPA
```

```
my_dict["cgpa"] = <your CGPA>
print("After adding cgpa:", my_dict)

# iii. Increase age by 1
my_dict["age"] += 1
print("After increasing age:", my_dict)

# iv. Delete branch in two separate copies
dict_pop = my_dict.copy()
returned_value = dict_pop.pop("branch")
print("Using pop():", dict_pop, "| Returned:", returned_value)

dict_del = my_dict.copy()
del dict_del["branch"]
print("Using del:", dict_del)
# pop() removes and returns the value; del removes the key/value
# but does not return the removed value.

# v. Iterate using items()
for key, value in my_dict.items():
    print(f"{key} → {value}")

# vi. Safe check for a missing email key
if "email" in my_dict:
    print("Email:", my_dict["email"])
else:
    print("Email key is not present.")

# vii. Friend dictionary with the same five original keys
friend_dict = {
    "name": "<friend name>",
    "roll_no": "<friend roll number>",
    "branch": "<friend branch>",
    "age": <friend age>,
    "city": "<friend home city>"
}

merged = {**my_dict, **friend_dict}
print("Merged dictionary:", merged)
# When both dictionaries have the same key, the value from friend_dict
# wins because it is unpacked second.

# viii. Keep only string-valued pairs from the ORIGINAL dictionary
original_dict = {
    "name": "<your name>",
    "roll_no": "<your roll number>",
    "branch": "<your branch>",
    "age": <your age>,
    "city": "<your home city>"
}
string_values = {k: v for k, v in original_dict.items() if isinstance(v, str)}
print("String-valued pairs:", string_values)
```

• The city key is renamed using pop(), which works with an existing dictionary rather than manually recreating it.

• pop() returns the removed value; del simply deletes the item.

• items() provides each key and its corresponding value during iteration.

• The in operator checks key existence before dictionary access, preventing a KeyError for a missing email key.

• With {**my_dict, **friend_dict}, duplicate keys take the value from the dictionary unpacked later.

• The final comprehension filters the original dictionary using isinstance(v, str), so only string-valued entries remain.

Submission note

Before submitting, replace every placeholder such as YOUR_ROLL_NUMBER, , , , and with your actual details. Run the program in Python and paste the resulting outputs if your instructor requires screenshots/output.

Source basis: UCS420 Cognitive Computing — Assignment 2, covering Lists, Tuples, Sets and Dictionaries.